Basics of Linux OS

1. Linux Prompt

In Linux, a prompt is what you see in the terminal before you type a command. It provides key information about your current shell session.

Example:

```
[root@server ~]#
[user@server ~]$
```

Breakdown:

- `root` → Current logged-in user.
  - `root` is the superuser or administrator account with full privileges to control the entire system.
- `server` → The hostname of the machine. Used to identify the computer within a network.
- `~` (tilde) → Current working directory.
  - `~` means the user's home directory.
- `#` (hash) → Indicates the user is the root (administrator) and has full privileges.
- `$` (dollar) → Indicates the user is a regular user with limited privileges. To execute admin-level commands, you usually use `sudo`.

---

2. Basic Linux Commands

Here are some essential Linux commands and their usage:

2.1 User and System Info

- `whoami` → Displays the current logged-in username.
- `hostname` → Displays the system's machine (host) name.

- **`hostnamectl`** → Shows detailed hostname and system information. Can be used to set system name.
- **`exec bash`** → Reloads the current shell, applying updated configurations without restarting the terminal.

2.2 Files and Directories Management

- **`ls`** → Lists files and directories in the current directory.
  - Options: `-l` for detailed list, `-a` for hidden files.
- **`touch file.txt`** → Creates a new empty file or updates timestamp of existing file.
- **`mkdir dir_name`** → Creates a new directory.
  - Use `-p` to create parent directories if they don't exist.
- **`cat file.txt`** → Displays file content; can also combine multiple file contents.
- **`history`** → Shows previously executed commands.
  - Use **`history -c`** to clear history.
- **`clear`** → Clears terminal screen.
- **`date`** → Displays or sets system date and time.
- **`cal`** → Displays a calendar.
  - Example: **`cal 8 2025`** (August 2025)
- **`head file.txt`** → Displays the first 10 lines of a file. Use `-n` to specify a number.
- **`tail file.txt`** → Displays the last 10 lines of a file. Use `-n` to specify a number.
- **`uname`** → Shows system information. Use `-a` for all details.

2.3 Disk and Memory Utilities

- **`free`** → Displays RAM and swap memory usage.
- **`du`** → Shows disk usage of files and directories.
- **`df`** → Shows free disk space. Use `-h` for human-readable sizes.
- **`file filename`** → Identifies the file type.
- **`tree`** → Shows directory structure in a tree-like format.

2.4 File and Directory Deletion

- **`rm filename`** → Deletes a file.
- **`rmdir dir_name`** → Deletes an empty directory.
- **`rm -rf dir_name`** → Deletes a directory and its contents forcefully.

- `-r` → Recursive delete
- `-f` → Force delete without prompt

2.5 Command Options Syntax

```
command [options] [arguments]
```

Example:

```
ls -l /home/user
```

- command → `ls`
- option → `-l` (detailed list)
- argument → `/home/user` (directory to list)

---

3. Command History Shortcuts

- Up Arrow → Browse previous commands one by one.
- !<number> → Re-run a specific command from history based on ID number.

---

4. Linux Editors

Popular editors:

- gedit → GUI-based text editor.
- vi / vim → Command-line text editors.

4.1 Modes in vim

- Insert Mode → Used for typing text.
  - Enter insert mode: `i, a, o`
- Command Mode → Used for executing editor commands. Enter by pressing `Esc`.
- Execute Mode → Directly issues commands like save, quit, replace.

4.2 Useful vim Commands

In Command mode:

- `:set number` → Show line numbers.
- `:set nonumber` → Hide line numbers.
- `:/pattern` → Search for a string.
- `:noh` → Remove search highlight.
- `:w` → Save file.
- `:q` → Quit without saving.
- `:wq` → Save and quit.
- `:saveas new_name` → Save file as new name.
- `:%s/pattern/replace/g` → Search and replace all occurrences.

In Execute/Normal mode:

- `yy` → Copy line.
- `p` → Paste.
- `nyy` → Copy n lines.
- `yiw` → Copy a word.
- `dd` → Delete line.
- `ndd` → Delete n lines.
- `x` → Delete character.
- `u` → Undo.
- `G` → Go to end of file.
- `gg` → Go to top of file.
- `n+G` → Jump to nth line from bottom.
- `n+gg` → Jump to nth line from top.
- `ZZ` → Save and quit.

---

5. Linux File Hierarchy

Linux directories are organized as per Filesystem Hierarchy Standard (FHS).

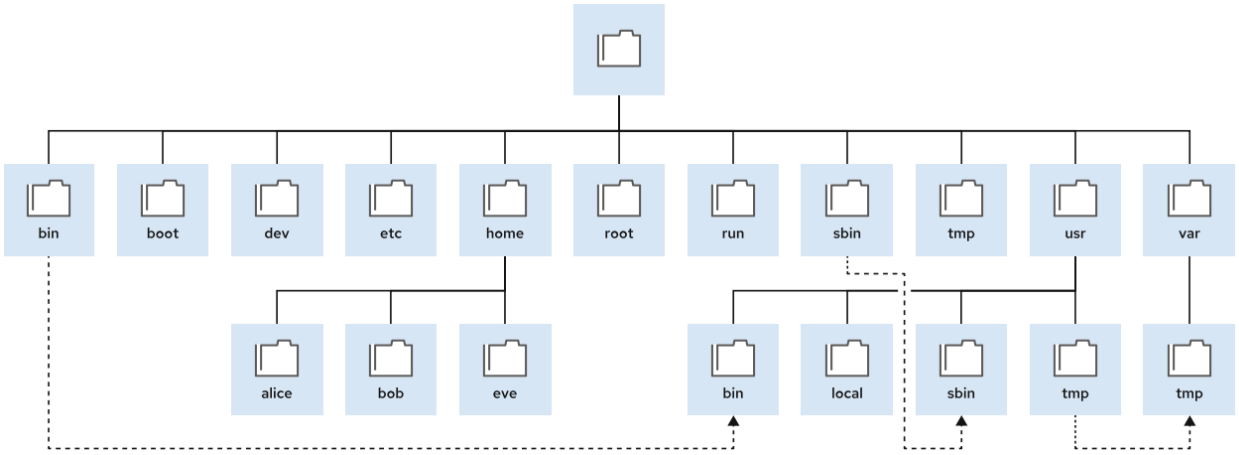
Main Directories:

- `/` → Root directory, top of hierarchy.

- `/bin` → Essential command binaries.
- `/sbin` → System binaries (admin commands).
- `/boot` → Boot loader and kernel files.
- `/dev` → Device files for hardware.
- `/etc` → Configuration files.
- `/root` → Home directory of `root`.
- `/home` → User home directories.
- `/mnt` → Temporary mount points.
- `/tmp` → Temporary files.
- `/srv` → Data for system services.
- `/usr` → User programs and libraries.
- `/var` → Variable data (logs, mail, database).
- `/lib` → Shared libraries.
- `/proc` → Process and kernel info (virtual filesystem).

---

6. Linux Paths



Types of Paths:

- Absolute Path → Starts with `/`. Full path from root.
  - Example: `/home/user/file.txt`
- Relative Path → Relative to current directory.
  - Example: `docs/file.txt`

Special Symbols:

- `.` → Current directory.
- `..` → Parent directory.
- `~` → Current user's home.

---

7. Copy, Move, Rename

Copy (cp)

```
cp [options] source destination
```

Examples:

- Single file: `cp file.txt /path/`
- Multiple files: `cp file1 file2 /path/`
- Directory: `cp -r dir /path/`
  Options:
- `-v` → Verbose
- `-f` → Force overwrite
- `-p` → Preserve attributes
- `-n` → No overwrite

Move / Rename (mv)

```
mv [options] source destination
```

Examples:

- Move file: `mv file.txt /path/`
- Rename file: `mv old.txt new.txt`
- Move directory: `mv dir /path/`

---

8. Viewing Large Files

- `more file.txt` → View file one page at a time (limited backward navigation).

- `less file.txt` → View file interactively (scroll up/down, search).

Linux Users and Groups

---

1. Introduction

- Linux is a multi-user operating system, meaning multiple human users and system processes (daemons) can operate on the same system simultaneously.
- Users: Entities (human or process) that can interact with the OS.
- Groups: Collections of users used for easier permission management.
- This structure enhances security, access control, and organization.

---

2. Users in Linux

What a User Account Contains

Each user account is uniquely identified by:

- Username → Human-readable name.
- UID (User ID) → Numerical ID for the user (used internally by the system).
- Home Directory → Private file storage space.
- Shell → Default command-line environment.

Types of Users

1. Root User

- UID = 0.
- Full system access and permissions.
- Can read/modify system files, install/remove software, configure hardware, and start/stop services.
- Used for administration tasks — *should not be used for daily work*.

2. Regular User

- UID range: 1000+ (varies by distro).
- Created for human users to perform daily, non-critical tasks.
- Limited to their home directory and given files.
- Requires sudo to run admin commands.

3. System User *(Expanded Explanation)*

- UID range: 1–999 (or similar, depending on distro).
- Represents background services rather than humans (e.g., `mysql`, `www-data`, `postfix`).
- No login shell (`/usr/sbin/nologin` or `/bin/false`).
- Purpose:
  - Run services securely with limited privileges.
  - Isolate service processes for security (if one is compromised, others remain safe).
  - Manage access to files related to that service only.
- Example Use Cases:
  - `mysql` → owns database files and processes.
  - `www-data` → runs Apache/Nginx web server processes.
  - `sshd` → manages SSH remote sessions.

---

3. Where User Information is Stored

- /etc/passwd
  - Stores account details (username, UID, GID, home, shell, comment field).
  - Password field is typically `x` (indicating it's stored in `/etc/shadow`).
  - Example:

```
jack:x:1001:1001:Akshay:/home/jack:/bin/bash
```

  -
  - Breakdown:
    - `jack` → username
    - `x` → password in `/etc/shadow`
    - `1001` → UID
    - `1001` → primary GID
    - `jack` → comment/full name
    - `/home/jack` → home directory
    - `/bin/bash` → default shell

- /etc/shadow
    - Stores encrypted passwords and password aging.
    - Example:

```
jack:$6$hashvalue:19236:0:99999:7:::
```

    -

---

4. Groups in Linux

- Definition: A group is a collection of users used to manage permissions collectively.
- Each group is identified by:
    - Group Name
    - GID (Group ID)
    - Members (usernames)

Types of Groups

1. Primary Group
    - Assigned when a user is created.
    - When the user creates a file, it is owned by their primary group.
2. Secondary Groups
    - Additional groups the user is a member of for accessing shared resources.
    - A user can belong to multiple secondary groups.

Where Group Information is Stored

- /etc/group → Stores group info.
- /etc/gshadow → Stores encrypted group passwords and admin settings.

Example from **`/etc/group`**:

```
developers:x:1002:akash,vaibhav
```

---

5. User Management Commands

Task	Command Example
Add User	`useradd username`
Add User + Password	`useradd jack` → `passwd jack`
View Users	`cat /etc/passwd`
Modify User	`usermod`
Add to Secondary Group	`usermod -aG groupname username`
Change Shell	`usermod -s /bin/zsh username`
Delete User	`userdel username`
Delete User + Home Dir	`userdel -r username`

---

6. Group Management Commands

Task	Command Example

Add Group	`groupadd developers`
View Groups	`cat /etc/group`
Add User to Group	`usermod -aG developers vaibhav`
Change Group Name	`groupmod -n newname oldname`
Delete Group	`groupdel developers`
Set Group Admin	`gpasswd -A username groupname`
Switch to Group	`newgrp groupname`

---

7. Advanced User & Group Management

- Custom Home Directory:
  `useradd -d /opt/users/john john`
- Account Expiry Date:
  `useradd -e 2024-12-31 john`
- Assign Custom UID or GID:
  - `useradd -u 2001 username`
  - `groupadd -g 1050 groupname`
- Lock/Unlock Accounts:
  - Lock: `passwd -l username`
  - Unlock: `passwd -u username`

---

8. UID (User ID) in Linux

- Definition: Unique numerical ID for a user. The system uses UID internally instead of username.
- UID Ranges:
  - 0 → root
  - 1–99 → reserved for predefined system accounts
  - 100–999 → system/service accounts (non-human)
  - 1000+ → regular human users
- Use in System:
  - File ownership
  - Process execution permissions
  - Access control

---

9. How UID and GID Work Together

- UID → identifies the user.
- GID → identifies the primary group.
- Together → determine ownership and permissions.

---

10. Best Practices

- Use groups to manage permissions instead of per-user changes.
- Avoid using root account daily; instead use `sudo`.
- Assign system users for services rather than running them as root.
- Regularly audit `/etc/passwd` and `/etc/group` for unused accounts.

---

11. Practice Questions

1. Command to create new user `john`.
2. Where is Linux user account info stored?
3. Difference between primary and secondary group?
4. Command to add `john` to `developers` group.
5. Purpose of `/etc/shadow`.
6. Create user `mike` with home at `/opt/mike`.
7. Lock/unlock user `alan`.

8. Purpose of `gpasswd`.
9. UID difference between 0 and 1000.
